

We introduce a validity-preserving hierarchical framework for switch placement and logical routing in Electronic Design Automation

Motivation

- Floorplan design is an extremely complex and costly task. Automating part of it enables faster iteration cycles and can lead to more efficient designs. In this work, we focus on switch placement and logical routing.
- In standard workflows, physical wirelength is optimized first, followed by logical routing. This sequential optimization can lead to suboptimal designs.
- Joint optimization is challenging: solutions must satisfy hard constraints, while most of the unconstrained search space is invalid. As a result, naive search methods spend most of their compute exploring infeasible designs.
- We propose a first step toward scalable joint optimization of switch placement and logical routing. Although our experiments consider much smaller instances than industrial designs, they demonstrate the potential of the framework.

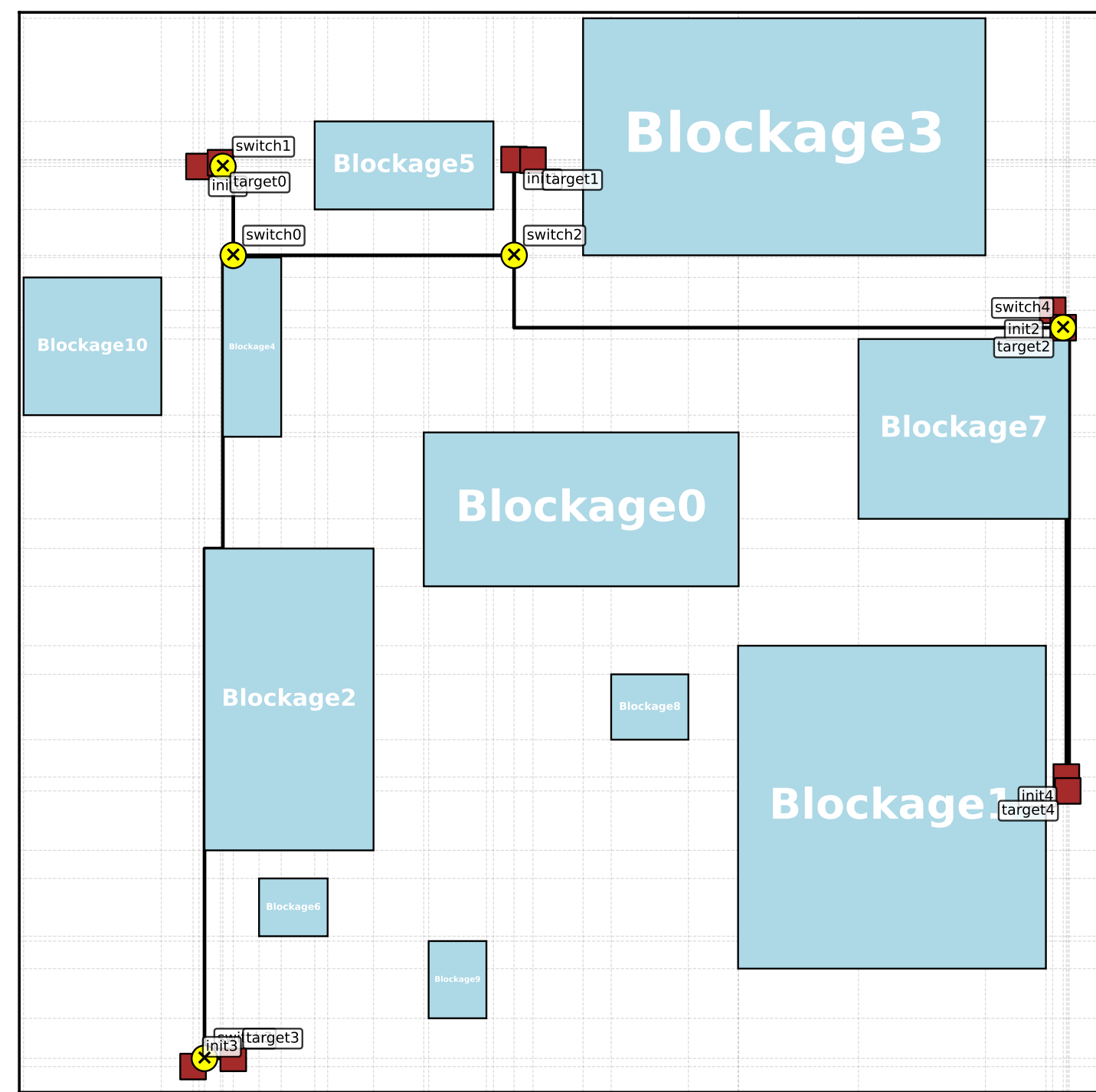
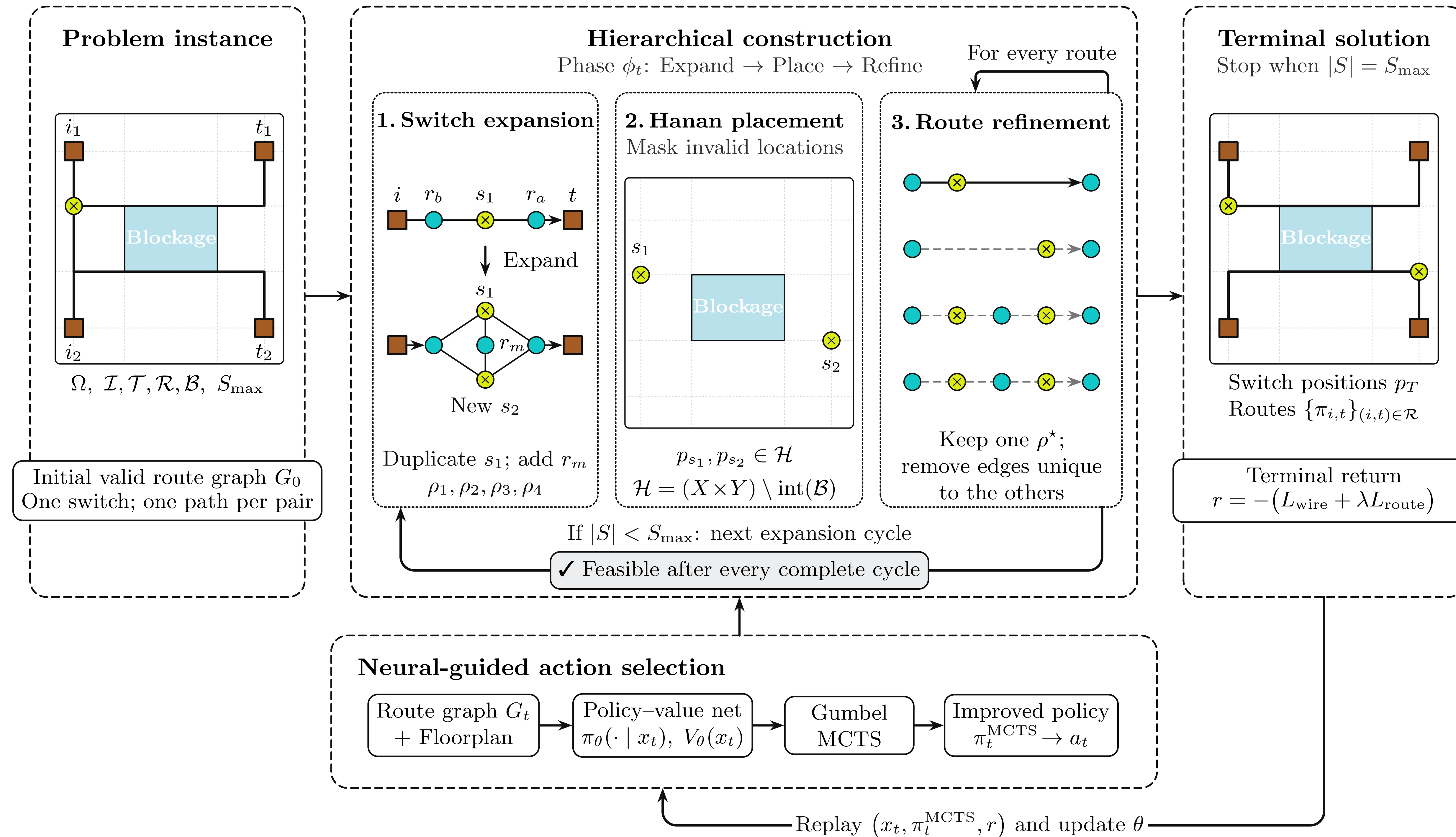


Figure. An example of a problem instance.

Pipeline



Ideas

We model the task as a combinatorial optimization problem on graphs. Our objective is to find a graph that minimizes $R = L_{\text{wire}} + \frac{1}{2} L_{\text{route}}$. Our three main ideas are:

- Route nodes:** Instead of representing routes as edge attributes, we explicitly model them as nodes in the graph. Allowing or forbidding a route to traverse an edge then simply amounts to connecting or disconnecting the corresponding route nodes.
- Hanan placement:** Newly expanded switches can be restricted to intersection points of the extended Hanan grid without losing optimality.
- Validity preservation:** After each complete expansion–placement–refinement cycle, each route is assigned a single loop-free path. This restricts the search to feasible floorplans without losing optimality.

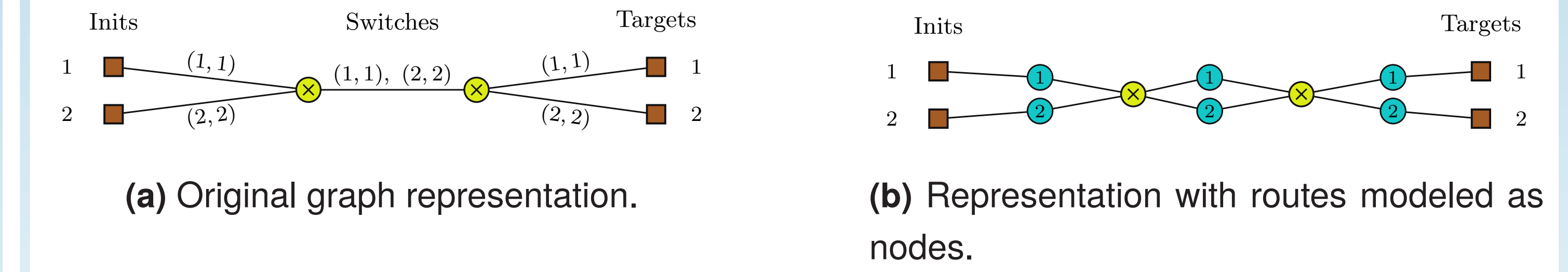
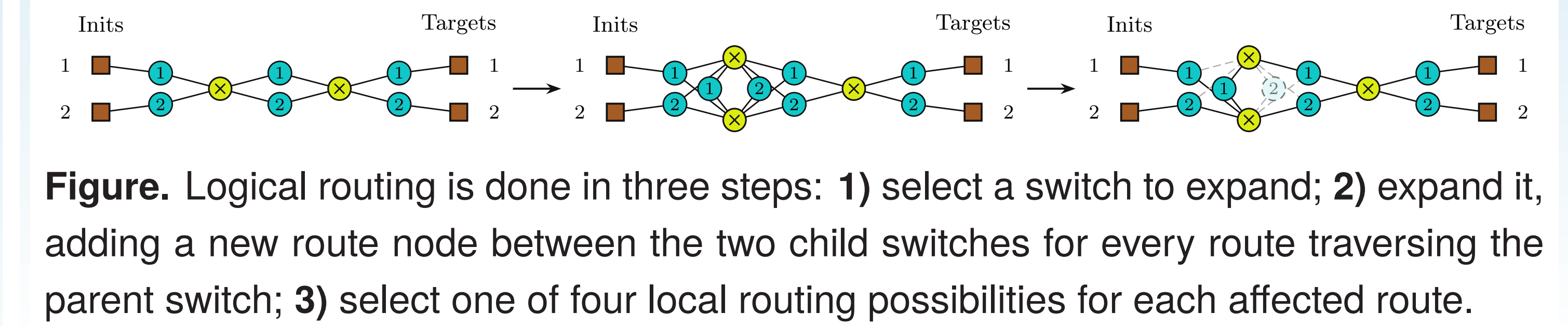
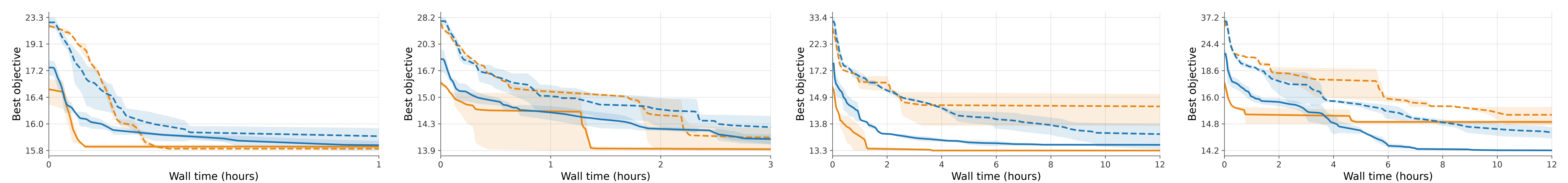


Figure. We explicitly represent routes in the graph using a dedicated type of node.



Results

Method	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Heuristic	11.902	11.255	15.349	11.330	14.847	11.126	8.090	10.301	11.292	14.615	14.741	13.418	13.662	15.003	14.512	15.284	16.571	16.366	16.277	16.277	14.909	15.415	14.007	15.684
Random search	18.706	20.799	24.042	17.420	23.629	19.310	12.672	18.574	18.663	24.931	22.584	20.775	21.425	21.875	19.338	20.146	20.943	30.810	29.030	20.421	23.683	21.337	22.457	20.583
Genetic algorithm	14.382	13.999	16.854	12.622	17.970	13.107	8.224	11.954	14.152	15.799	16.478	16.504	15.238	15.057	14.134	15.236	15.239	19.881	22.062	15.726	16.580	14.531	14.621	15.277
PPO-EWMA	12.166	11.484	15.670	11.576	14.717	<u>10.960</u>	8.595	10.352	<u>11.142</u>	<u>13.329</u>	<u>14.151</u>	13.361	13.287	<u>13.593</u>	14.134	<u>15.269</u>	15.239	19.147	18.488	<u>15.817</u>	<u>13.095</u>	13.989	<u>12.409</u>	15.277
Gumbel MCTS	11.333	10.880	14.274	9.784	13.926	9.918	7.739	9.897	10.820	13.028	13.995	13.361	13.287	13.466	14.134	15.236	15.239	13.601	14.373	15.726	12.779	13.471	12.386	15.277



Experiments

We pretrain a policy on a dataset of 24 random floorplans using *PPO-EWMA* or *Gumbel MCTS*. The left table presents our results and compares them against classical optimization baselines (*Random Search*, *Genetic Algorithm*), as well as a custom Steiner-inspired *Heuristic* that iteratively places switches based on their marginal contribution to the objective. The table reports the objective for the 24 floorplans in the pretraining set. Learnable search improves over the classical baselines, and Gumbel MCTS in particular outperforms all of them.

We then use the pretrained policy as an initialization on four held-out floorplan instances, and compare the optimization time required to reach convergence with similar policies initialized from scratch. The four plots each correspond to one held-out floorplan and report the best objective found as a function of wall time. The y-axis is shown on a logarithmic scale for readability. Blue lines denote Gumbel MCTS, and orange lines denote PPO-EWMA; solid lines correspond to pretrained initialization, while dotted lines correspond to scratch initialization. Pretraining consistently accelerates convergence, showing that the policy can transfer knowledge learned on seen floorplans to new ones and reach better solutions faster.

Acknowledgments: The authors acknowledge the French National Research Agency (ANR) for its financial support of the System On Chip Design leveraging Artificial Intelligence (SODA) project under grant \NR-23-IAS3-0004 and the JCJC project DeSNAP ANR-24-CE23-1895-01. This work is also supported by Hi! PARIS and ANR/France 2030 program ANR-23-IACL-0005.

Conclusions

- We propose a novel hierarchical approach for jointly optimizing switch placement, physical routing, and logical routing.
- Our formulation restricts the search space to feasible solutions, without losing optimality.

Next: Scale the framework to industrial-size problems and incorporate more realistic modeling, objectives, and constraints.